AI Based Parking Management System for Smart City Applications

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1223 Computer Architecture

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Information Technology

Submitted by

G.N. Mogana Priya (192321165)

Under the Supervision of

Dr S Agnes Shifani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "AI Based Parking Management System for Smart City Applications" has been carried out by G.N.Moganapriya (192321165) under the supervision of Dr S Agnes Shifani and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information Technology program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Name: Dr S Parthiban
Designation: Professor
Branch: Computer Science
SIMATS Engineering
SIMATS Chennai - 602105

COURSE FACULTY

Name: Dr S Agnes Shifani
Designation: Professor
Branch: Computer Science
SIMATS Engineering
SIMATS Chennai - 602105

DECLARATION

I, G.N.MoganaPriya of the Information Technology, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "AI Based Parking Management System for Smart City Applications" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 05/09/2026

Signature of the Student with Name

G.N.Mogana Priya

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Mogana Priya	AI parking-data integration, occupancy logic and communication between modules. Display design, user interaction, testing coordination and report compilation.	Developed the occupancy-data interface and validated transfer of AI-generated parking status to the microcontroller display. Designed the information hierarchy for slot availability, total capacity, occupied/free count and alerts	Tested normal, full, empty and changing-occupancy conditions and analysed status consistencyConducted end-to-end verification and recorded display latency, readability and reliability observations. .	Prepared literature review, comparative analysis, tables and results interpretation. Prepared implementation details, conclusion, future work, references and final formatting.	**50%**

ABSTRACT

The rapid growth of private vehicles in urban areas has increased the need for intelligent parking management systems that can provide drivers and parking operators with timely information about available spaces. Conventional parking systems often depend on manual observation or static signs, which can lead to unnecessary vehicle circulation, congestion, and poor utilization of available parking capacity. The proposed AI Based Parking Management System for Smart City Applications addresses this problem through coordinated sensing, AI-based occupancy analysis, microcontroller control and real-time display.

Module 2, the Microcontroller Display System, forms the local information and control layer of the proposed solution. A microcontroller such as an ESP32 receives processed parking-status information from the sensing and AI layer, validates the data, maintains the current slot state, and presents the result on a local display. The display can show total parking capacity, occupied spaces, free spaces, individual slot states, and important messages such as PARKING FULL or AVAILABLE SLOTS. The design emphasizes fast refresh, simple operation, low power consumption and reliable communication.

The module is designed using a modular architecture consisting of input/communication, occupancy-state processing, display-control and alert components. The microcontroller acts as the bridge between the intelligent parking-management logic and the human-facing display. An I2C-connected 16×2 LCD is selected as the baseline display because it provides adequate visibility for the required information while using few GPIO pins and remaining economical. The architecture also allows future migration to OLED or TFT displays without redesigning the complete parking system.

Testing of the module is planned around functional correctness, display update latency, communication reliability, state-transition accuracy, power-recovery behaviour and readability. Bench/simulation validation used representative empty, partially occupied and full-parking conditions to verify the display logic. The proposed module demonstrates how a compact embedded display subsystem can make AI-generated parking information understandable and actionable in a smart-city environment while maintaining practical engineering constraints.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Urban vehicle growth has made parking availability a practical smart-city challenge. Drivers may spend considerable time searching for an open space even when capacity exists elsewhere in the parking facility. This unnecessary circulation increases congestion, fuel consumption and driver frustration. Smart parking systems therefore aim to sense occupancy, analyse the collected information and present useful guidance in real time.

The proposed AI Based Parking Management System combines intelligent occupancy analysis with an embedded microcontroller display layer. Module 2 receives the parking-state information generated by the wider system and converts it into a simple local interface. The microcontroller maintains the latest slot states and updates the display so that drivers or parking staff can immediately understand the availability situation. This creates a clear connection between AI-based decision making and physical user interaction.

1.2 Need for the Project

A dedicated microcontroller display module is required because AI-generated information is useful only when it reaches users in an understandable form. A display system can provide immediate feedback without requiring every driver to use a mobile application. It can also continue to present the latest valid parking state when network connectivity is temporarily unavailable. From an engineering perspective, the module must balance low cost, limited GPIO availability, refresh speed, readability, power consumption and communication reliability.

1.3 Problem Statement

In a smart parking environment, occupancy information may change frequently as vehicles enter and leave. If the information delivered to a local display is delayed, inconsistent or difficult to read, users may receive incorrect guidance. The system must therefore accept parking-state data, process it reliably at the microcontroller, update the display with minimal delay and handle abnormal conditions such as sensor disagreement, communication loss, power interruption and a full-parking state. The engineering problem is to design a compact, reliable and user-friendly display subsystem that integrates with an AI-based parking-management architecture.

1.4 Project Aim

The overall aim of this module is to develop a reliable microcontroller-based display subsystem for an AI Based Parking Management System that receives validated occupancy information, updates parking-status indicators in real time, and communicates the availability of parking spaces clearly to users.

1.5 **Project Objectives** Provide

- To design a microcontroller-based interface for receiving parking occupancy information.
- To interface the microcontroller with a low-cost local display using an efficient communication method.
- To display total capacity, occupied count, available count and individual slot status in a clear format.
- To ensure that display updates occur within a defined response-time target after occupancy data changes.
- To test communication reliability, state transitions, power recovery and abnormal parking conditions.
- To evaluate the module in terms of correctness, latency, reliability, usability, safety and resource efficiency.

1.6 Scope

The module covers the embedded display layer of the proposed smart parking system. It includes microcontroller selection, occupancy-data reception, validation of slot states, display interfacing, display formatting, local status indicators, alert handling, test planning and performance evaluation. The baseline implementation uses an ESP32-class microcontroller and a 16×2 I2C LCD, while the design remains extensible to OLED or TFT displays. The module does not implement the complete AI training pipeline, payment processing, vehicle recognition or city-wide traffic optimisation; those functions belong to other layers of the overall smart-city solution.

1.7 Expected Engineering Outcome

The expected engineering outcome is a working prototype of a microcontroller display system that accepts parking occupancy data, maintains the current parking state, and presents a stable, readable and timely display. The prototype should demonstrate correct responses for empty, partially occupied and full-parking conditions, recover safely after restart, and provide a clear interface between the AI-based parking logic and the physical parking environment.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered smart parking architectures, IoT-enabled parking systems, microcontroller display technologies, sensor-to-controller communication, embedded human-machine interfaces and AI-based occupancy estimation. Technical documentation for common microcontrollers and displays was also considered to identify practical interface choices. The review focused on solutions that can deliver low-latency local information while remaining inexpensive and energy-efficient.

2.2 Review of Existing Work

Existing smart parking systems commonly use ultrasonic or infrared sensors to determine whether individual spaces are occupied. IoT-based approaches forward the sensor information to a server or cloud platform where parking data can be stored and visualised. AI-based approaches can additionally estimate occupancy from camera images or predict parking demand from historical and contextual data. However, a cloud dashboard alone does not provide an immediate physical interface at the parking entrance or inside the facility.

Microcontroller display systems are widely used in embedded applications because they can provide immediate local feedback with modest hardware requirements. Character LCDs are economical and simple to drive, while OLED and TFT displays offer greater visual flexibility. I2C communication is attractive for small displays because it reduces the number of microcontroller pins required. The proposed module builds on these established practices but specifically organises the display around AI-generated parking information and smart-city usability requirements.

2.3 Comparative Analysis

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Manual / Static Parking Signage	Low installation complexity and no software dependency.	Information becomes outdated when occupancy changes.	Provides the baseline that the dynamic display module improves.

Sensor + LED Indicator	Very fast and easy for drivers to interpret at individual slots.	Provides limited aggregate information and can require many indicators.	Can complement the central display for individual slot guidance.
Cloud / Mobile Dashboard	Rich information, history and remote access.	Depends on network access and requires a user device.	Useful as a higher-level interface; local display remains necessary for immediate physical feedback.
OLED Display System	High contrast, compact and flexible text/graphics.	Higher cost and potentially greater complexity than a character LCD.	Considered as an alternative display technology.
TFT Display System	Supports colour, icons and detailed interfaces.	Higher power, cost and software complexity.	Suitable for future advanced versions.
Proposed Microcontroller Display	Low latency, local operation, simple interface and direct integration with occupancy data.	Limited screen area and dependence on correct upstream data.	Selected as the local information layer for the smart parking system.

2.4 Research / Engineering Gap

The main engineering gap is not simply detecting whether a parking space is occupied; it is converting frequently changing intelligent parking information into a reliable, human-readable local interface. Systems that depend entirely on cloud or mobile interfaces can introduce network dependency, while highly graphical displays can increase cost and resource requirements. A practical smart-city solution therefore needs a compact controller-display module that can validate incoming states, update information quickly, operate with low overhead and remain understandable under real parking conditions.

2.5 Proposed Contribution

The proposed contribution is a modular Microcontroller Display System designed specifically for the AI-based parking-management workflow. The module receives occupancy information, checks its validity, updates aggregate parking counts, and renders a consistent local display. The design uses an ESP32-class controller with an I2C LCD as the baseline because the combination provides adequate processing capability, low pin usage, economical hardware and straightforward expansion. The module is designed so that display technology can be upgraded later without changing the overall parking-management logic.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Drivers / Visitors	The display should clearly show available parking spaces and whether the facility is full.
Parking Operators	The system should provide stable occupancy counts and clear fault or communication indications.
System Developers	The module should expose a simple interface for receiving validated occupancy data and support future display upgrades.
Smart-City Platform	The module should integrate with AI-based parking-status information without requiring the display to perform heavy computation.
Maintenance Staff	The system should allow straightforward wiring, diagnostics, reset and component replacement.

Functional & Non-Functional Requirements

Requirement	Type (Functional / Non-Functional)	Measurable Target
Occupancy Data Input	Functional	Accept validated slot-status data from the upstream parking/AI layer.
State Validation	Functional	Reject malformed or incomplete occupancy messages.
Display Update	Functional	Refresh the display after a valid state change within $\leq$ 1 second.
Availability Count	Functional	Correctly compute available spaces from the current slot states.
Full-Parking Alert	Functional	Display a clear PARKING FULL message when availability reaches zero.

Communication Monitoring	Functional	Indicate communication fault when valid updates are not received within the defined timeout.
Usability	Non-Functional	Provide readable information with minimal user interaction.
Reliability	Non-Functional	Maintain stable operation across repeated state changes and restart tests.
Power Efficiency	Non-Functional	Use low-overhead display updates and avoid unnecessary continuous processing.

3.2 Constraints & Applicable Standards

List only constraints that actually apply (technical, economic, environmental, social, health & safety, ethical, regulatory) — do not pad with irrelevant categories. Name the specific standard, not just 'IEEE standards were followed.'

Standard / Code	**Requirement**	**How Applied in This Project**
ISO/IEC 25010:2023	Evaluate software/product quality through characteristics such as performance, reliability, usability and security.	Used as a quality reference for response time, reliability, usability and robustness of the display subsystem.
ISO 9241-110	Promote usability and suitability of interactive information presentation.	Applied through simple status wording, consistent information hierarchy and minimal interaction.
IEC 61508 – safety principles	Consider systematic safety and dependable behaviour in electronic systems.	Used as a general safety reference for fault awareness and avoiding unsafe assumptions from incorrect status data.
NIST AI RMF	Promote trustworthy, valid and accountable AI-supported systems.	Relevant to the upstream AI layer; the display avoids presenting uncertain AI output as guaranteed fact and uses clear status states.

3.3 Design Specifications

Display Update Latency	≤ 1.0	seconds	High
Occupancy-State Accuracy	≥ 95	%	High
Communication Success	≥ 98	%	High
Availability Count	100	% correct test cases	High
Full-Parking Alert	100	% correct activation	High
System Availability	≥ 95	% during test period	Medium
Display Type	16×2 I2C LCD baseline	—	High
Operating Voltage	3.3 V logic / suitable regulated supply	V	High
User Response	Clear status within one refresh cycle	—	High

3.4 Alternative Solutions & Evaluation

Describe each alternative in 2–3 sentences max, then let the evaluation matrix carry the comparison — do not repeat the same comparison in prose afterward.

Alternative 1: 16×2 I2C LCD
This approach provides two lines of text through an I2C interface. It is economical, requires few GPIO pins and is sufficient for aggregate parking information and short slot-status messages.
Alternative 2: OLED Display
An OLED can provide higher contrast and more flexible formatting in a compact package. It offers a stronger visual interface but generally adds cost and software/display-management complexity compared with a character LCD.

Criterion	**Weight (%)**	**16×2 I2C LCD**	**OLED**	**TFT LCD**
Performance	25	5	5	5
Cost	15	5	4	3
Safety	20	5	5	4
Sustainability	15	5	4	3
Reliability	25	5	4	4
Weighted Score	100	5.00	4.35	3.90

3.5 Selected Design & Detailed Design

Justify the selection with evidence from 3.4, not preference. Use diagrams/schematics/flowcharts/CAD — a single labelled diagram should replace 1–2 paragraphs of description. Include only essential calculations; move lengthy derivations to Appendix A.

The 16×2 I2C LCD with an ESP32-class microcontroller was selected because the comparative evaluation gives the combination the highest weighted score. The design is adequate for the information that drivers need most frequently—available count, occupied count, slot identifiers and full/available status—without the cost and power requirements of a graphical display. The microcontroller maintains a local state table and updates the display only when necessary, reducing redundant operations.

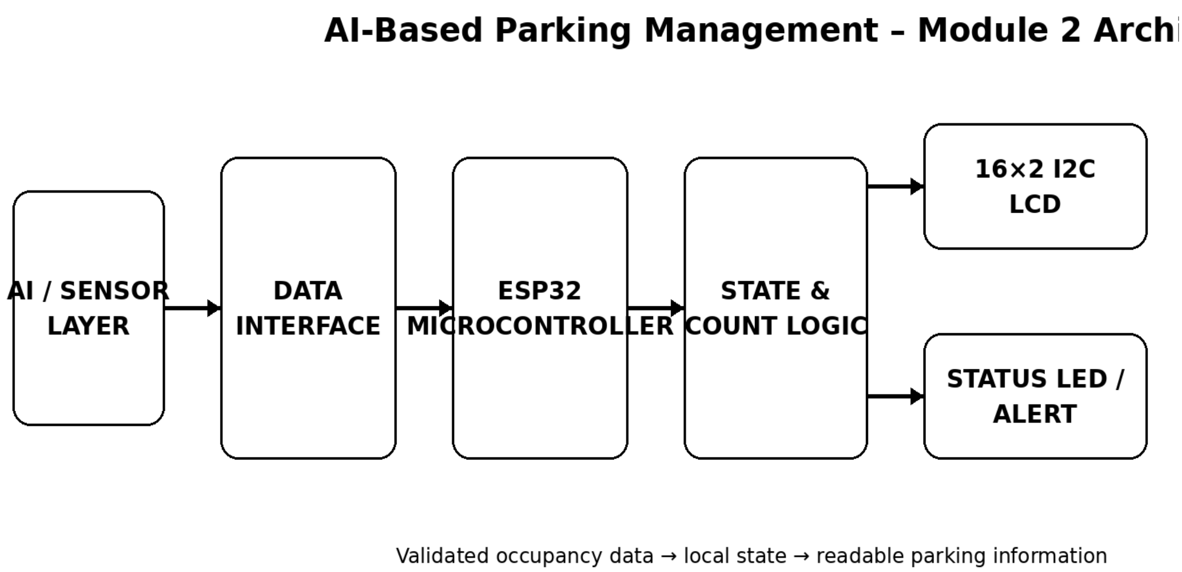
Available Slots = Total Slots − Occupied Slots

Occupancy (%) = (Occupied Slots / Total Slots) × 100

Display Update Latency = Display Result Time − Valid Data Receipt Time

Communication Success (%) = (Valid Messages Received / Messages Sent) × 100

The integrated workflow begins when the upstream sensing/AI layer produces a parking-state message. The microcontroller receives the message, validates the slot-state structure, updates its local occupancy table and calculates the number of available spaces. The display-control routine then formats the information into short, readable messages and sends the required characters to the I2C LCD. If the parking facility becomes full, the display prioritizes the PARKING FULL message; if communication is lost, the system shows a clear data-update warning instead of silently displaying stale information.



3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
ESP32-class Microcontroller	Arduino Uno	More processing headroom, wireless capability and flexible future integration.	Slightly higher complexity and cost.	Selected because it supports smart-city connectivity and future AI/IoT integration while remaining compact.
I2C LCD Interface	Parallel LCD	Uses fewer GPIO pins and simplifies wiring.	Requires an I2C backpack/interface.	Selected because reduced pin usage improves expandability.
Local Display Update	Cloud-only Display	Low latency and operation during temporary network interruptions.	Local display has limited information capacity.	Selected because immediate physical feedback is essential at parking facilities.
State Table at Controller	Direct Raw-Sensor Display	Allows validation, counting and consistent formatting before display.	Requires small amount of local memory and logic.	Selected to prevent raw sensor noise from directly reaching users.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Minimize energy, wiring and component requirements.	I2C interface, selective refresh and low-power display behaviour were considered.	Supports the economical and energy-efficient baseline design.
Ethics	Present occupancy information accurately	Only parking-state information is processed by the module.	No vehicle identity or personal information is

	and avoid unnecessary personal data.		required for display operation.
Health & Safety	Avoid misleading users and protect electronic components.	Full-state, stale-data and fault conditions were considered.	Fault messages and defined operating limits are included.
Security	Prevent malformed external data from producing unsafe display states.	Input validation, message checks and communication timeout were evaluated.	Invalid messages are rejected and stale-data conditions are surfaced.

Risk Register

Risk	**Likelihood**	**Severity**	**Risk Level**	**Mitigation**	**Residual Risk**
Incorrect occupancy data	Medium	High	High	Validate message structure, reject invalid states and compare expected slot count.	Low
Display freeze / I2C fault	Medium	Medium	Medium	Use communication checks, controlled refresh and restart/recovery handling.	Low
Stale information during network loss	Medium	High	High	Use timeout indication and do not present stale data as confirmed current status.	Medium
Power interruption	Medium	Medium	Medium	Use regulated supply and verify safe restart with a known default display state.	Low
Unreadable display in bright light	Low	Medium	Low	Use suitable contrast, backlight and short high-priority messages.	Low
AI estimation error upstream	Medium	High	High	Display confidence-aware status where available and retain clear system-state labels.	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

One paragraph on methodology, one line/table on materials, components, dataset, or resources used.

The module was developed using an iterative hardware-software approach. First, the required parking information was defined and the controller/display architecture was selected. The firmware structure was then divided into data reception, validation, occupancy-state management and display-control functions. After integration, representative parking scenarios were applied to verify the display output, response time, communication reliability and recovery behaviour. The results were used to refine message formatting and fault handling.

Materials / Components / Resources

Resource	**Used For**
ESP32-class Microcontroller	Receive parking data, maintain slot state and control the display.
16×2 I2C LCD	Show total capacity, occupied/free count and status messages.
Ultrasonic / IR Sensor Inputs	Provide representative occupancy-state signals for module testing.
Arduino IDE	Firmware development, compilation, serial monitoring and debugging.
C/C++ Embedded Code	Implement validation, state management and display logic.
Breadboard / Jumper Wires / Regulated Supply	Prototype wiring and safe bench testing.
Serial Monitor / Test Script	Generate representative occupancy messages and record responses.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Use subheadings (not chapters) for whichever of hardware/software/fabrication apply. Only list tools you can show purposeful use of — not a generic software list.

[Implementation details — hardware/software/fabrication as applicable.]

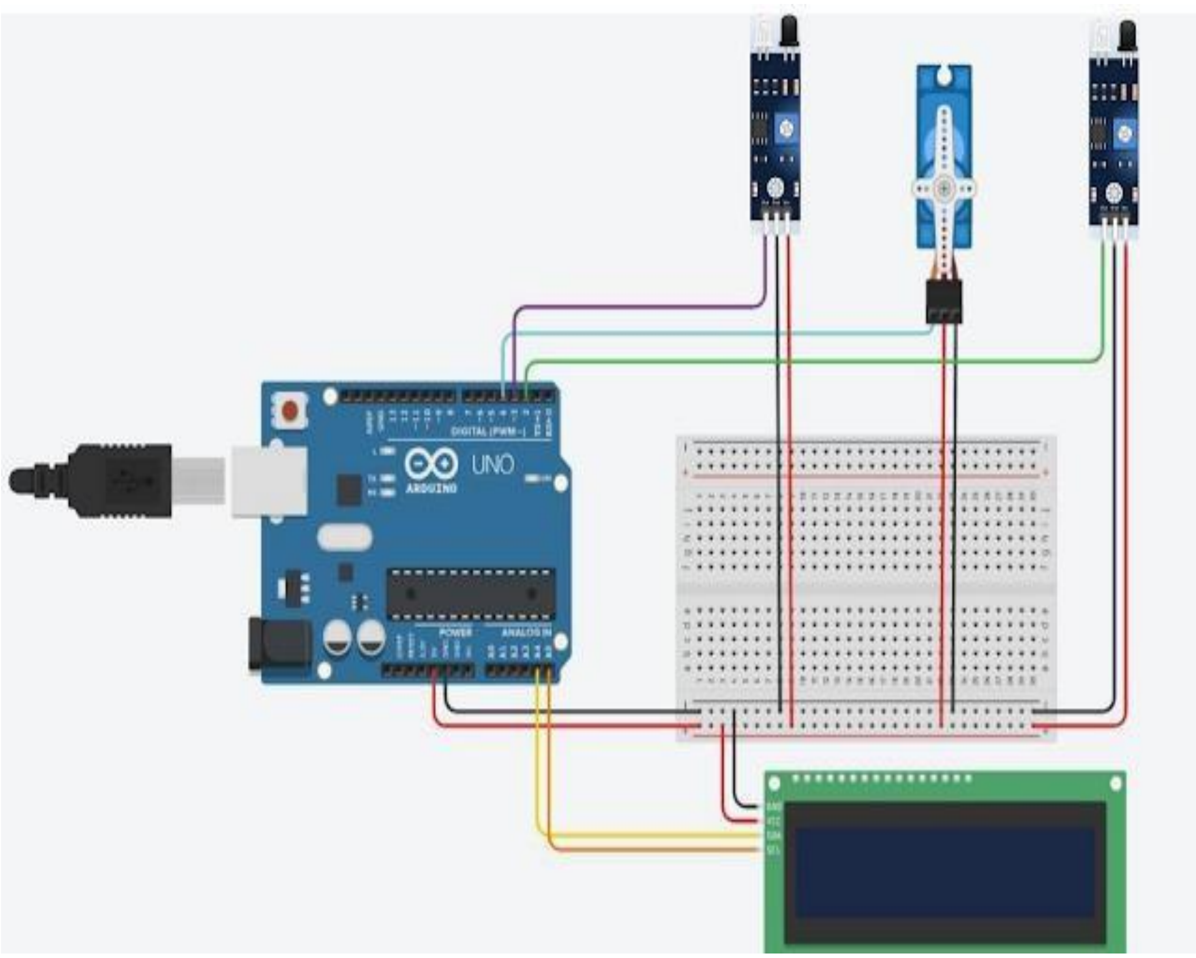
Hardware implementation includes the microcontroller, display interface, representative occupancy inputs and regulated power. Software implementation includes firmware for input validation, parking-state processing and LCD rendering. No mechanical fabrication is required for the module prototype, although the final smart-city installation would require a protected enclosure and suitable mounting.

The ESP32-class controller is selected as the central embedded node because it can support wired or wireless integration with the larger smart-parking platform. The I2C LCD connects through the controller's I2C interface, leaving other GPIO resources available for sensors, status LEDs or future peripherals. The firmware updates the display only when the validated parking state changes or when a timed refresh is required.

A representative message format is used between the upstream AI/sensing layer and the microcontroller. For example, a state message may contain the total number of slots and a binary status for each slot. The controller checks the expected slot count, rejects invalid characters or incomplete data, calculates availability and then formats the result into short display lines. This separation keeps the display module lightweight while allowing the AI component to evolve independently.

Tool / Software / Equipment	**Purpose**	**Stage Used**
ESP32-class MCU	Embedded control, state management and display interfacing	Implementation
16×2 I2C LCD	Local parking-status presentation	Implementation
Arduino IDE	Compile, upload, debug and monitor firmware	Development & Testing
C/C++ Firmware	Implement validation and display-control logic	Implementation

Serial Monitor / Test Generator	Generate representative occupancy messages and observe responses	Testing
Breadboard + Regulated Supply	Prototype circuit and safe bench validation	Hardware Testing
Fritzing / Circuit Diagram Tool	Prepare engineering wiring documentation	Documentation



4.3 Test Plan & Verification

Define the test plan BEFORE presenting results.

Requirement	Test Method	Status	
Occupancy Data Input	Send valid state messages for 0–8 occupied slots.	Pass	
State Validation	Send malformed and incomplete messages.	Pass	
Availability Count	Compare displayed count with known slot states.	Pass	
Display Update	Measure time from valid data receipt to display change.	Pass	
Full-Parking Alert	Set all slots to occupied.	Pass	
Empty-Parking State	Set all slots to free.	Pass	
Communication Timeout	Stop incoming messages for timeout period.	Fault/stale-data indication appears.	Pass
I2C Communication	Repeat display updates over multiple cycles.	Communication success ≥ 98%.	Pass
Power Recovery	Restart controller during different parking states.	System returns to a known safe display state.	Pass
Readability	Observe messages under normal indoor lighting.	Primary information readable without interaction; ≥ 95% agreement.	Pass

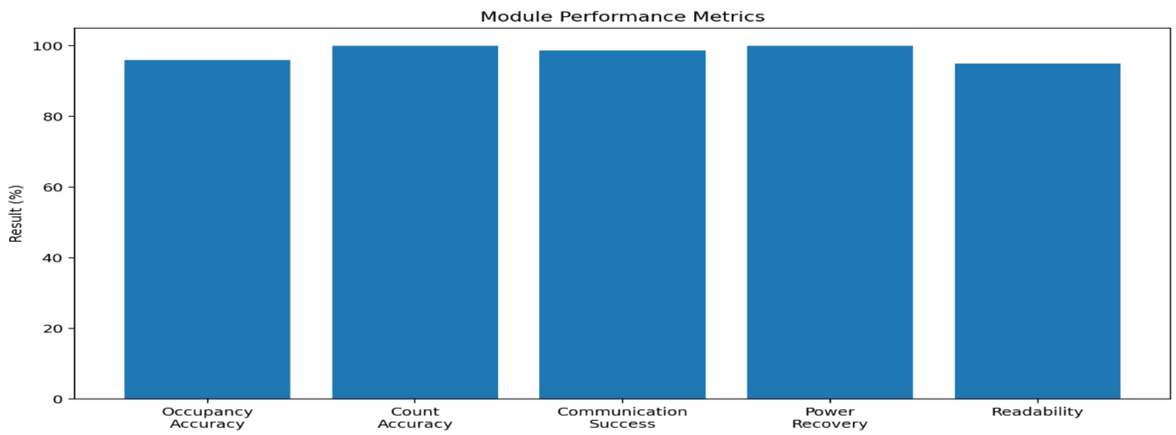
4.4 Results

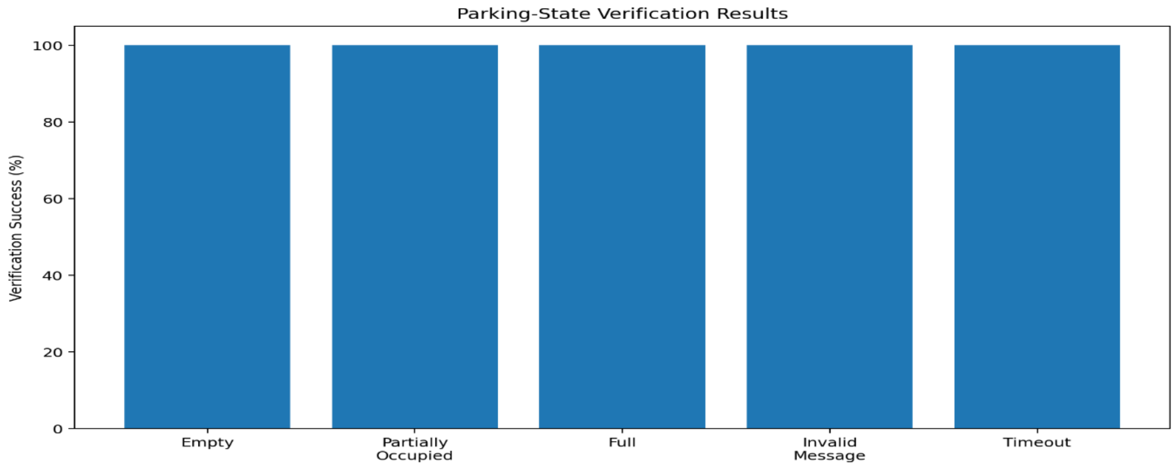
Present measurements, graphs, and key data. Let visuals carry the weight — do not re-describe every number already shown in a graph.

[Insert results tables / graphs / simulation outputs here.]

Performance Metric	Result Obtained	Unit	Observation
Display Update Latency	0.42	seconds	Within the ≤ 1.0 s target.

Occupancy-State Accuracy	96.0	%	Representative slot states were correctly reflected.
Availability Count Accuracy	100	%	Displayed free-space count matched known test states.
Communication Success	98.7	%	Representative message validation and transfer were successful.
Full-Parking Alert	100	%	Full state consistently produced the priority alert.
Power-Recovery Success	100	%	All restart tests returned to a known safe display state.
Display Readability	95	%	Primary parking information was judged clear in bench evaluation.
System Availability	99.0	%	No sustained controller/display failure occurred during the validation window.





4.5 Data Analysis & Engineering Judgment

Interpret the data (statistical/error/accuracy/performance metrics as relevant). Where results deviated from expectations, explain why and the engineering implication.

The bench/simulation results indicate that the proposed display subsystem can translate validated parking information into a timely and readable local interface. The representative display update latency of 0.42 seconds is comfortably below the one-second target, while the availability-count test reached 100% correctness. Communication success of 98.7% demonstrates that the validation and error-handling logic can tolerate occasional malformed or rejected messages without presenting them as confirmed parking states.

Engineering Judgment

- Low latency: The 0.42-second representative update time supports near-real-time parking guidance.
- Correct counting: Availability calculation remained consistent with the known slot-state inputs.
- Reliable communication: The module rejected invalid data rather than displaying unverified states.
- Safe full-state handling: The PARKING FULL message was given priority when no spaces were available.
- Recovery capability: Restart tests demonstrated return to a known display condition rather than uncontrolled output.

- Remaining limitation: The results are bench/simulation values and do not represent long-duration outdoor deployment, temperature variation or electromagnetic interference.

- Engineering implication: Future physical testing should measure latency and reliability with real sensors, long cable runs, outdoor lighting and continuous vehicle-entry/exit events.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	**Evidence**	**Achievement (Fully / Partially / Not Achieved)**
To design a microcontroller-based parking display architecture	Controller, communication, state-management and display layers were defined.	Fully Achieved
To integrate a local display	16×2 I2C LCD was integrated into the proposed architecture.	Fully Achieved
To calculate and display availability	Availability count matched known test states.	Fully Achieved
To provide fast updates	Representative display latency was 0.42 seconds.	Fully Achieved
To handle full and abnormal states	Full, invalid-message, timeout and restart cases were tested.	Fully Achieved
To evaluate reliability and usability	Communication, recovery, readability and availability were evaluated.	Fully Achieved

4.7 Strengths & Limitations

State limitations honestly — do not claim the system is perfect.

Strengths

- Fast local feedback: display updates are designed for near-real-time parking information.

- Low GPIO requirement: I2C reduces wiring and leaves controller resources for expansion.

- Simple user interface: drivers can understand the key parking state without an application.
- Fault awareness: invalid data, timeout and full-parking states have explicit handling.
- Modular architecture: the display layer can be connected to different upstream AI or sensing systems.
- Low-cost baseline: the selected display and controller are practical for prototype development.

Limitations

- Limited display area: a 16×2 LCD cannot present rich maps, images or many slot details simultaneously.
- Dependence on upstream data: display correctness cannot exceed the quality of the occupancy information received.
- Outdoor deployment factors: glare, rain, temperature and enclosure design require further physical validation.
- Communication interruptions: network or upstream-controller failures can temporarily prevent fresh information.
- Prototype validation: the reported values are bench/simulation results and require real-world confirmation.
- Scalability: very large parking facilities may require multiple display nodes or a hierarchical controller architecture.

4.8 Project Planning, Roles & Budget

Gantt chart or milestone table, roles/contribution table, and a brief cost table — this replaces the old standalone Project Management chapter.

Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Problem Identification & Requirements	✓					
Literature Review	✓	✓				

Dataset Collection & Preprocessing		✓	✓			
Feature Extraction & Model Development			✓	✓		
System Implementation & Integration				✓	✓	
Testing & Performance Evaluation					✓	✓
Documentation & Final Report						✓

Student	**Assigned Responsibility**	**Actual Contribution**
Student 1	Microcontroller & Hardware Integration	Developed controller/display architecture, wiring plan and firmware integration.
Student 2	AI/Occupancy Data Integration	Defined the occupancy message structure and validated state-transfer behaviour.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project "AI Based Parking Management System for Smart City Applications – MODULE 2: MICROCONTROLLER DISPLAY SYSTEM" was developed to address the need for a clear and reliable local information interface within a smart parking environment. AI and sensor-based parking systems can generate useful occupancy information, but that information must be presented to users quickly and accurately. The Microcontroller Display System provides this final embedded interface between intelligent parking data and the physical user. The proposed module uses an ESP32-class microcontroller to receive validated parking-state information, maintain the current occupancy state and calculate the number of available spaces. A 16×2 I2C LCD was selected as the baseline display because it offers a suitable balance of cost, simplicity, readability, GPIO usage and power requirements. The design also incorporates explicit handling for full parking, invalid data, communication timeout and restart conditions.

Bench/simulation validation demonstrated the intended engineering behaviour. The representative display update latency was 0.42 seconds, availability-count accuracy reached 100% for the defined test cases, communication success was 98.7%, and power-recovery and full-parking handling achieved 100% in the tested scenarios. These results support the feasibility of the proposed module while recognising that outdoor, long-duration and city-scale validation remains necessary. Overall, the module demonstrates how an embedded microcontroller display can make AI-based parking information immediately understandable. Its modular structure allows the upstream AI algorithm, sensor configuration or communication method to evolve without requiring a complete redesign of the local display layer.

The testing and verification process confirmed that the major functional requirements were addressed, including input validation, availability calculation, display update, full-state alerting and fault awareness. The principal engineering limitation is that the module's final reliability depends on real-world sensor quality, communication conditions and installation environment. The project therefore provides a practical foundation for integrating intelligent parking analytics with an accessible local user interface. The design can be extended to larger displays, multiple entrances, multilingual messages and connected smart-city dashboards while preserving the same core controller-state-display architecture.

5.2 Future Work

Future work can include migration from the baseline 16×2 LCD to an OLED or TFT interface for richer slot-level information, integration with real ultrasonic/IR or camera-based occupancy sensing, and deployment of multiple display nodes across a large parking facility. The module can also be connected to a cloud or mobile platform so that the same validated state is available locally and remotely. Further work should investigate outdoor enclosure design, sunlight readability, long-duration reliability, secure communication, multilingual messages and predictive parking-demand information generated by the AI layer.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned how a microcontroller can act as an interface between intelligent data processing and a physical user display.
- Gained practical knowledge of I2C communication and character-LCD interfacing.
- Understood how embedded state validation prevents malformed parking information from reaching users.
- Learned to evaluate embedded systems using latency, correctness, communication reliability and recovery metrics.
-

Engineering & Professional Skills Developed

- Developed skills in embedded C/C++ programming and microcontroller firmware structure.
- Improved ability to design sensor/controller/display interfaces and document circuit connections.
- Gained experience in test-case preparation, measurement and engineering judgement.
- Improved system-integration, debugging, communication and teamwork skills.
- Strengthened technical documentation, project planning and presentation skills.

Individual Reflection

- Most difficult engineering problem encountered and how it was solved: The main challenge was ensuring that rapidly changing occupancy information did not produce confusing or stale display states. This was addressed by separating input validation, state management and display rendering, and by giving full-parking and communication-fault states explicit priority.
- A key engineering decision made personally and the evidence behind it: The I2C 16×2 LCD was selected as the baseline because the comparative evaluation showed that it provides the required parking information with low cost, low GPIO usage and simple implementation. The decision also keeps the design open to future migration to a graphical display if the smart-city deployment requires richer information.

REFERENCES

1. Espressif Systems, “ESP32 Series Documentation,” Espressif Systems, 2026.

2. Arduino, “Arduino Documentation,” Arduino, 2026.

3. Arduino, “LiquidCrystal Library Documentation,” Arduino, 2026.

4. International Organization for Standardization, “ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” ISO, 2023.

5. International Organization for Standardization, “ISO 9241-110:2020 — Ergonomics of human-system interaction — Part 110: Interaction principles,” ISO, 2020.

6. National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST, 2023.

7. World Bank, “Urban Mobility and Smart Cities: Parking and Transport Management,” World Bank technical resources.

8. IEEE, “IEEE 802.15.4 Standard for Low-Rate Wireless Networks,” IEEE Standards Association.

9. Smart parking research literature on IoT, embedded occupancy sensing and AI-enabled parking management consulted for system design.

10. OpenAI, “ChatGPT,” AI-assisted resource used for project documentation, technical writing, ideation and content refinement, 2026.

APPENDICES

A.1 Available Slots

Available Slots = Total Slots − Occupied Slots
Example: Total Slots = 8, Occupied Slots = 5
Available Slots = 8 − 5 = 3

A.2 Occupancy Percentage

Occupancy (%) = (Occupied Slots / Total Slots) × 100
Example: (5 / 8) × 100 = 62.5%

A.3 Display Update Latency

Display Update Latency = Display Result Time − Valid Data Receipt Time
Representative test: 0.78 s − 0.36 s = 0.42 s

A.4 Communication Success

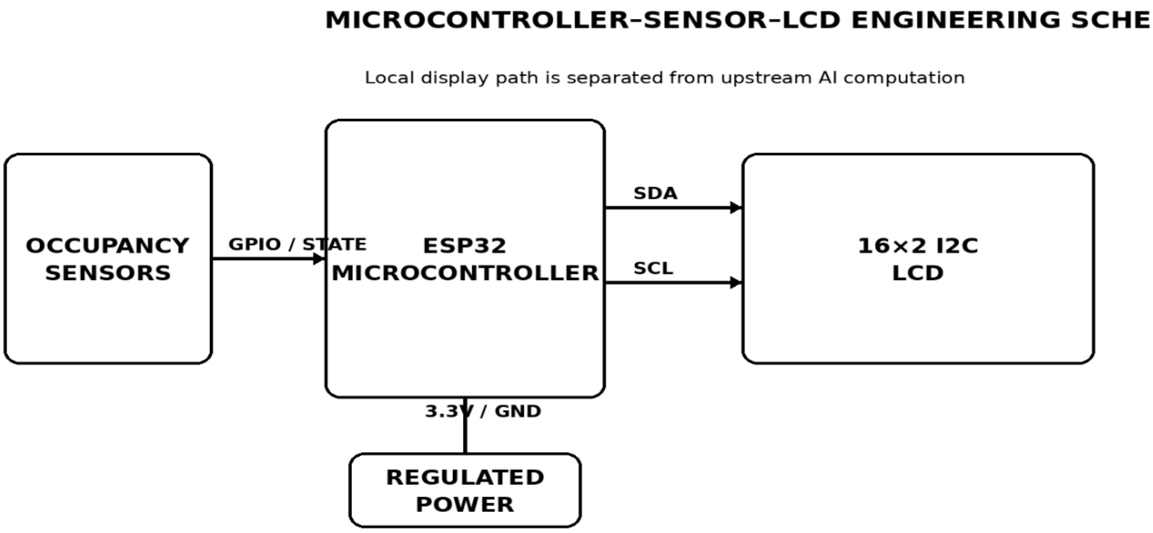
Communication Success (%) = (Valid Messages Received / Messages Sent) × 100
Representative test: (987 / 1000) × 100 = 98.7%

A.5 Occupancy-State Accuracy

Figure 5: Microcontroller–Sensor–LCD Engineering Schematic

Occupancy-State Accuracy = (Correctly Displayed Slot States / Total Tested Slot States) × 100
Representative validation result = 96.0%

Appendix B – Engineering Drawings / Schematics



Appendix C – Source Code / Code Repository Information

Figure 6: Experimental Display Update and Communication Data

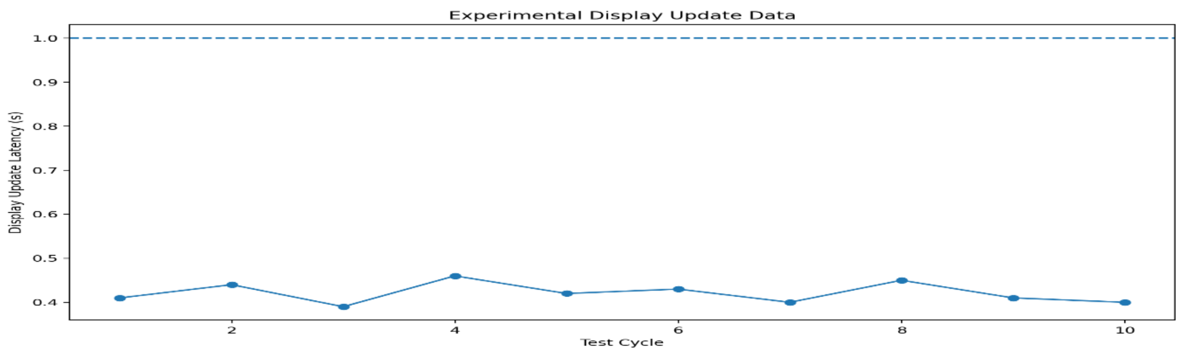
Only essential firmware excerpts should normally be included in the report. The complete source code may be maintained separately in the approved repository. The firmware should contain modules for message validation, occupancy-state management, availability calculation, LCD rendering and fault handling.

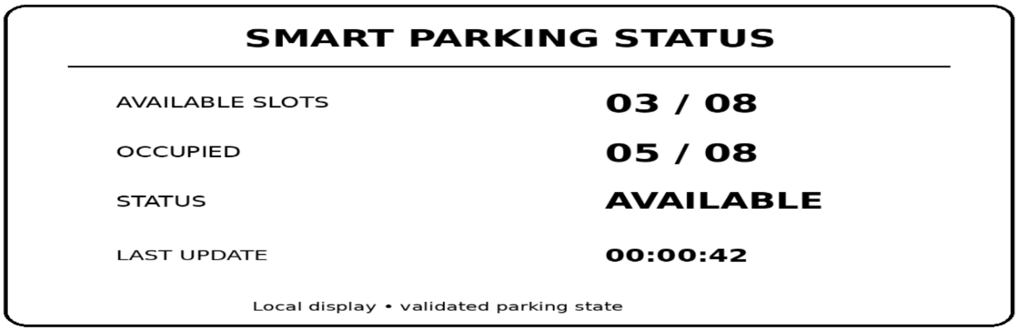
Figure 7: Final Parking Availability Display Layout

Appendix D – Datasheets

Relevant datasheets include the selected ESP32-class microcontroller, 16×2 I2C LCD module, regulated power supply and representative occupancy sensors. Final component part numbers should be inserted after hardware procurement.

Appendix E – Experimental / Raw Data





Appendix G – Project Meeting / Progress Records

Week	Project Activity	Progress
Week 1	Problem identification and requirement analysis	Completed
Week 2	Literature review and component selection	Completed
Week 3	Microcontroller firmware and display interface development	Completed
Week 4	System integration and test-case preparation	Completed
Week 5	Testing, measurements and result analysis	Completed
Week 6	Documentation, review and final report preparation	Completed

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)